# Ders 9 Çalışma Özeti

## Ders 9 Çalışma Özeti

### Genel Konular

- Dersin tarihi 4 Mayıs 2021 Salı. Ders Zoom üzerinden, İngilizce yapıldı.
  - Asıl anlatım yaklaşık 00:06:40'ta başlar. Ders vize haftasından sonraki ilk derstir.
  - Araya iki ara verildi (yaklaşık 00:49:30-01:00:40 ve 01:55:10-02:07:00).
- Bağlamdan bağımsız diller için pompalama önsavı (pumping lemma for context-free languages)
  - Önsavın ifadesi: $z = uvwxy$ bölünmesi ve üç koşul.
  - $L = \{0^i 1 0^i 1 0^i\}$ dilinin bağlamdan bağımsız olmadığının ispatı.
  - $L_1 = \{0^i 1 0^i\}$ (bağlamdan bağımsız), $L_2 = \{ww\}$ (değil), $L_3 = \{ww^R\}$ (bağlamdan bağımsız) karşılaştırması.
- Bağlamdan bağımsız gramerlerin sadeleştirilmesi (simplification of CFGs)
  - Yerine koyma (substitution) kuralı.
  - Boşa giden değişkenlerin (nullable variables) ve $\varepsilon$-üretimlerinin kaldırılması.
  - Birim üretimlerin (unit productions) kaldırılması.
  - Yararsız üretimlerin (useless productions) kaldırılması: önce terminal üretemeyen, sonra başlangıçtan ulaşılamayan değişkenler.
- Normal biçimler (normal forms)
  - Chomsky normal biçimi (CNF, Chomsky normal form) ve dönüştürme adımları.
  - Greibach normal biçimi (GNF, Greibach normal form) ve basit bir dönüştürme örneği.
- Araya giren duyurular: vize ve Quiz 2 sonuçları ders duvarına yüklendi. Vize ortalaması yaklaşık 50-55.

### Hocanın Özellikle Vurguladığı Kısımlar

- Pompalama önsavının ifadesini yazın ve ezberleyin.
  - Önsavın ispatı derste yapılmadı. Hoca ispatı kitaptan okumanızı istedi.
- Bağlamdan bağımsız diller en fazla iki sayımı eşleştirebilir.
  - Bir desende üç sayının eşit olması gerekiyorsa ($0^n 1 0^n 1 0^n$ gibi), dil büyük olasılıkla bağlamdan bağımsız değildir. Bunu ispatlamak için pompalama önsavı kullanılır.
- Pompalama önsavı yalnızca bir dilin bağlamdan bağımsız **olmadığını** ispatlamak için kullanılır.
  - Bir dizginin pompalanabildiğini göstermek, dilin bağlamdan bağımsız olduğunun ispatı değildir. Bağlamdan bağımsız olduğunu ispatlamak için o dili üreten bir gramer vermelisiniz.
- İki sayı eşleşiyor diye dili hemen bağlamdan bağımsız saymayın.
  - $\{ww\}$ ilk bakışta iki şeyi eşleştiriyor gibi görünür, ama bağlamdan bağımsız değildir.
- İspatın sonunda çelişkiyi açıkça yazın.
  - "Hiçbir $uvwxy$ bölünmesi pompalanamıyor. Bu bir çelişki. O hâlde $L$'nin bağlamdan bağımsız olduğu varsayımı yanlış."
- CNF insan için değil, bilgisayar için kullanışlıdır.
  - Ayrıştırma (parsing) ve teorem ispatlarında kullanılır.
- Kitabın 2. bölümünün alıştırmalarını ve paylaşılan çalışma kâğıdının çoğunu artık yapabiliyor olmalısınız.
  - Bölümde yalnız aşağı itmeli otomatlar (pushdown automata) kaldı.

## Kısa Tekrar Notları

- **CFL pompalama önsavı:** Her bağlamdan bağımsız dil $L$ için bir $n$ vardır. $|z| \geq n$ olan her $z \in L$, $z = uvwxy$ biçiminde yazılabilir ve:
  - $|vwx| \leq n$,
  - $|vx| \geq 1$,
  - her $i \geq 0$ için $uv^iwx^iy \in L$.
- **İspat şablonu:** $L$'yi bağlamdan bağımsız varsay. $n$ pompalama sabiti olsun. $n$'ye bağlı bir $z$ seç. Bütün bölünme durumlarında pompalanmış dizginin $L$'de olmadığını göster. Çelişki.
- **Yerine koyma:** $A \to xBz$ ve $B \to y_1$ ise, $A \to xy_1z$ eklenir. $B \to y_1$ silinir. $B$'nin başka üretimi yoksa $A \to xBz$ de değiştirilir.
- **Boşa giden değişken:** $X \Rightarrow^* \varepsilon$ ise $X$ boşa gidendir (nullable). Sağ taraflarda $X$'in olduğu ve olmadığı iki sürüm yazılır, $X \to \varepsilon$ silinir. Başlangıç değişkeninin $\varepsilon$ üretimi kaldırılamaz.
- **Birim üretim:** $X \to Y$ (tek değişkenden tek değişkene). $X \to X$ biçimindekiler hemen silinir.
- **Yararsız üretim:**
  - Adım 1: terminal dizgi üretebilen değişkenleri bul, diğerlerini içeren üretimleri sil.
  - Adım 2: bağımlılık grafiği (dependency graph) ile $S$'ten ulaşılamayan değişkenleri sil.
- **CNF:** Her üretim $A \to BC$ ya da $A \to a$ biçimindedir.
- **CNF'ye dönüştürme sırası:** $\varepsilon$-üretimlerini sil → birim üretimleri sil → (isteğe bağlı) yararsız üretimleri sil → terminaller için yeni değişkenler ($T_a \to a$) → uzun kuralları yeni değişkenlerle ($V_1, V_2, ...$) böl.
- **GNF:** Her üretim $A \to aV_1V_2 \cdots V_k$ ($k \geq 0$) biçimindedir: tek bir terminal ve ardından yalnız değişkenler.

## Detaylı Açıklamalar

### CFL pompalama önsavının ifadesi

Hoca önce düzenli diller için pompalama önsavını hatırlattı. Ardından bağlamdan bağımsız diller için yeni sürümü yazdı. Tahtadaki ifade şöyledir:

- Her bağlamdan bağımsız $L$ için bir $n$ tam sayısı vardır öyle ki, uzunluğu $n$'den büyük her $z \in L$ dizgisi $z = uvwxy$ biçiminde yazılabilir ve:
  1. $|vwx| < n$
  2. $|vx| > 0$
  3. Her $i > 0$ için $uv^iwx^iy \in L$.

Düzenli dillerdeki önsavdan farkı şudur: Bu kez **iki yer** ($v$ ve $x$) aynı anda pompalanır. Sınırların doğru biçimi ($\leq$, $\geq 0$) için "Ders sonrası notlar"a bakın.

### Örnek: $L = \{0^i10^i10^i\}$ bağlamdan bağımsız değildir

Tahtada dil $\{0^i10^i10^i \mid i > 1\}$ olarak yazıldı. Hoca önce sezgiyi verdi: Bu dilde üç blok sıfırın sayısı eşit olmalı. Bağlamdan bağımsız diller yalnız iki sayımı eşleştirebilir.

İspat (tahtadaki adımlar):

1. $L$'nin bağlamdan bağımsız olduğunu varsayalım (çelişki için).
2. $n$ pompalama sabiti olsun.
3. Desen: $z = 0^n10^n10^n$. Uzunluğu $n$'den büyüktür.
4. $z = uvwxy$ bölünmelerine bakılır:
   - **Durum 1:** $v$ ve $x$ birer $1$ olsun. 1'lerin sayısı sabit olduğu için pompalanamaz.
   - **Durum 2:** $v$ ilk sıfır bloğunda, $x$ son sıfır bloğunda olsun. Arada $n$ sıfır ve iki $1$ olduğu için $|vwx| < n$ sağlanmaz. Bu bölünme seçilemez.
   - **Durum 3:** $v$, $w$, $x$ ortadaki sıfır bloğunun içinde olsun. $i = 2$ için $0^n10^{n+2}10^n$ elde edilir. Bloklar eşit olmadığı için bu dizgi $L$'de değildir.
5. Hiçbir bölünme pompalanamıyor. Çelişki. $L$ bağlamdan bağımsız değildir.

Tahtadaki durumlar bütün olasılıkları kapsamıyor. Tam ispat için "Ders sonrası notlar"a bakın.

**$L_1$, $L_2$ ve $L_3$ karşılaştırması**

| Dil | Bağlamdan bağımsız mı? | Not |
|---|---|---|
| $L_1 = \{0^i 10^i \mid i > 1\}$ | Evet | İki sayım eşleşiyor. $z = 0^n 10^n$ için $v$ = 1'in solundaki son 0, $w = 1$, $x$ = sağındaki ilk 0 seçilirse $uv^2wx^2y = 0^{n+1}10^{n+1} \in L_1$. |
| $L_2 = \{ww \mid w \in (0+1)^*\}$ | Hayır | Tahtada seçilen dizgi $z = 0^n1^n0^n1^n$. Bu dizgi pompalanamaz. |
| $L_3 = \{ww^R \mid w \in (0+1)^*\}$ | Evet | Hoca, dizgilerin pompalanabildiğini kendiniz denemenizi istedi. |

Tahtadaki not: $L_1$ için yapılan pompalama, $L_1$'in bağlamdan bağımsız olduğunun ispatı **değildir**. İspat için bir gramer gerekir. Örnek gramer: $S \to 0S0 \mid 00100$.

**Yerine koyma kuralı**

Tahtadaki örnek:

| Başlangıç grameri | $B \to b$ yerine konunca | Ardından $B \to aA$ yerine konunca |
|---|---|---|
| $S \to aB$ | $S \to aB \mid ab$ | $S \to aaA \mid ab$ |
| $A \to aaA$ | $A \to aaA$ | $A \to aaA$ |
| $A \to abBc$ | $A \to abBc \mid abbc$ | $A \to abaAc \mid abbc$ |
| $B \to aA$ | $B \to aA$ | (silindi) |
| $B \to b$ | (silindi) | |

Üç gramer de aynı dili üretir. Bu Python ile 9 sembole kadar bütün dizgilerde denendi.

Genel kural (tahtadan):

- $A \to xBz$ ve $B \to y_1$ (terminal): $A \to xBz \mid xy_1z$ yazılır. $B$'nin başka üretimi ($B \to aC$) kaldığı için $A \to xBz$ korunur.
- $A \to xBz$ ve $B$'nin tek üretimi $B \to aC$: $A \to xaCz$ yazılır.

Derste bir öğrenci, ilk örnekte $B$'nin başka üretimi kalmadığı için $A \to xBz$'nin silinmesi gerektiğini sordu. Hoca haklı olduğunu söyledi. Ana fikir şu: Elde edilen gramer, orijinal gramerle aynı dizgileri üretmelidir.

**Boşa giden değişkenler**

- $X \to \varepsilon$ bir $\varepsilon$-üretimidir (null production). $X$ boşa giden bir değişkendir.
- Daha genel olarak $Y \Rightarrow \cdots \Rightarrow \varepsilon$ ise $Y$ boşa gidendir.

Örnek:

- Başlangıç: $S \to aMb$, $M \to aMb$, $M \to \varepsilon$.
- Sonuç: $S \to aMb \mid ab$, $M \to aMb \mid ab$.

İki gramer de $\{a^n b^n \mid n \geq 1\}$ dilini üretir.

Bütün $\varepsilon$-üretimleri kaldırıldıktan sonra, başlangıç değişkeni hariç bütün boşa giden değişkenler ortadan kalkar. $S \to \varepsilon$ kaldırılamaz.

**Birim üretimler**

Örnek: $S \to aA$, $A \to a$, $A \to B$, $B \to A$, $B \to bb$.

1. $A \to B$ yerine konur. Sağ tarafta $A$ görülen yerlere $B$ de eklenir: $S \to aA \mid aB$, $A \to a$, $B \to A \mid B$, $B \to bb$.
2. $B \to B$ hemen silinir.
3. $B \to A$ yerine konur: $S \to aA \mid aB \mid aA$. Tekrarlanan $aA$ silinir.
4. Son gramer: $S \to aA \mid aB$, $A \to a$, $B \to bb$.

İki gramer de $\{aa, abb\}$ dilini üretir.

Derste bir öğrenci, $B \to bb$'nin yanına bir $a$ eklenmesi gerekip gerekmediğini sordu. Hoca, değişikliklerin yalnız **sağ taraflarda** yapıldığını söyledi. Son durumda sağ tarafta $B$ kalmadığı için bir şey eklenmez.

**Yararsız üretimler**

Bazı türetimler hiç bitmez. Örnek: $A \to aA$ olup $A$'nın terminal üreten bir kuralı yoksa, $A$'yı kullanan türetim hiçbir zaman terminal dizgiye ulaşmaz.

Örnek gramer: $S \to aS \mid A \mid C$, $A \to a$, $B \to aa$, $C \to aCb$.

- **Adım 1 (yararlı değişkenler):**
  - 1. tur: Sağ tarafı yalnız terminal olan değişkenler: $A$, $B$.
  - 2. tur: Sağ tarafında yararlı değişken olanlar: $S$ ($S \to A$). Küme $\{S, A, B\}$.
  - $C$ yararsızdır. $C$'yi içeren üretimler silinir: $S \to aS \mid A$, $A \to a$, $B \to aa$.
- **Adım 2 (ulaşılabilirlik):** Bağımlılık grafiğinde düğümler değişkenlerdir. $S \to S$ ve $S \to A$ kenarları vardır. $B$'ye ulaşılamaz. $B$ silinir.
- **Sonuç:** $S \to aS \mid A$, $A \to a$.

**Chomsky normal biçimi**

- Her üretim ya $A \to BC$ (tam iki değişken) ya da $A \to a$ (tam bir terminal) biçiminde olmalıdır.
- $A \to aA$ gibi karışık kurallar CNF'de değildir.

Örnek: $S \to ABa$, $A \to aab$, $B \to Ac$.

1. **Terminaller için yeni değişkenler:** $T_a \to a$, $T_b \to b$, $T_c \to c$.
   - $S \to ABT_a$ (CNF değil), $A \to T_aT_aT_b$ (CNF değil), $B \to AT_c$ (CNF).
2. **Uzun kuralları bölme:** $V_1 \to BT_a$, $V_2 \to T_aT_b$.

Sonuç:

| Kural |
|---|
| $S \to AV_1$ |
| $A \to T_aV_2$ |
| $B \to AT_c$ |
| $V_1 \to BT_a$ |
| $V_2 \to T_aT_b$ |
| $T_a \to a$, $T_b \to b$, $T_c \to c$ |

İki gramer de yalnız $aabaabca$ dizgisini üretir (Python ile doğrulandı).

Genel dönüştürme sırası: $\varepsilon$-üretimlerini sil, birim üretimleri sil, isteğe bağlı olarak yararsız üretimleri sil, terminaller için değişken ekle, uzun kuralları böl.

**Greibach normal biçimi**

- Her üretim $A \to aV_1V_2 \cdots V_k$ ($k \geq 0$) biçimindedir.
- **GNF'de olan örnek:** $S \to cAB$, $A \to aA \mid bB \mid b$, $B \to b$.

- **GNF'de olmayan örnek:** $S \to abSb$, $S \to aa$. İlk terminalin ardından başka terminaller var.
- Dönüştürme: $T_a \to a$, $T_b \to b$ eklenir.
  - Sonuç: $S \to aT_bST_b$, $S \to aT_a$, $T_a \to a$, $T_b \to b$.
- Hocanın okuduğu nota göre GNF ayrıştırma için CNF'den de iyidir, ama bir gramerin GNF'sini bulmak CNF'sini bulmaktan zordur. Hoca bunun kendisine zor görünmediğini söyledi. Bu konuda "Ders sonrası notlar"a bakın.

**Duyurular**

- Quiz 1, vize ve Quiz 2 sonuçları ders duvarına yüklendi. Vize ortalaması yaklaşık 50-55.
- Bir öğrenci vize cevaplarının paylaşılıp paylaşılmayacağını sordu. Hoca, diğer grubun hocasıyla böyle bir şey konuşmadıklarını, şimdilik paylaşılmayacağını söyledi.
- Bir sonraki ders: aşağı itmeli otomatlar (pushdown automata).

**Ders sonrası notlar**

- **Önsavdaki eşitsizlikler:** Tahtada $|z| > n$, $|vwx| < n$ ve "her $i > 0$" yazıldı. Sipser'daki ve standart ifade şöyledir:
  - $|z| \geq n$,
  - $|vwx| \leq n$,
  - her $i \geq 0$ için $uv^iwx^iy \in L$.
  - $i = 0$ (aşağı pompalama) birçok ispatta gereklidir. Bu yüzden "$i > 0$" yazmayın.
- $|vx| > 0$ **şartı:** Hoca bunu sözlü olarak "$wx$'in uzunluğu sıfırdan büyük" diye okudu. Tahtadaki $|vx| > 0$ doğrudur. Pompalanan parçalar $v$ ve $x$'tir.
- **İspatın tamamlanması:** Tahtadaki üç durum bütün bölünmeleri kapsamaz. Tam gerekçe şöyledir:
  - $vx$ bir 1 içeriyorsa, $i = 0$ ya da $i = 2$ için 1'lerin sayısı 2 olmaktan çıkar. Dizgi $L$'de değildir.
  - $vx$ yalnız sıfırlardan oluşuyorsa, $|vwx| \leq n$ olduğu için $vwx$ en fazla iki komşu sıfır bloğuna değer. Üçüncü blok değişmez. $i = 2$ için blok uzunlukları eşit olmaz.
  - İki durumda da pompalanmış dizgi $L$'de değildir.
- **Pompalanmış dizginin yazımı:** Tahtada $uv^2wx^2z$ yazıldı. Son parça $z$ değil $y$'dir: $uv^2wx^2y$.
- $L_2$ **için dizgi seçimi:** Hoca sözlü olarak önce "$0^n10^n1$ gibi bir şey" dedi. Bu dizgi pompalanabilir: $v$ = ilk bloğun son sıfırı, $w = 1$, $x$ = ikinci bloğun ilk sıfırı seçilirse $0^{n+k}10^{n+k}1$ yine $ww$ biçimindedir. Doğru seçim tahtada yazılan $z = 0^n1^n0^n1^n$'dir.
- **Yerine koymada asıl ölçüt:** Kuralın değişkenli ya da terminalli olması değil, $B$'nin başka üretiminin kalıp kalmamasıdır. $B$'nin bir üretimi silinince, $B$'nin geçtiği her yere o üretimin sağ tarafı eklenir. $B$'nin hiç üretimi kalmazsa, $B$'yi içeren eski kurallar da silinir. Öğrencinin sorusu ve hocanın cevabı bununla uyumludur.
- **CNF ve boş dizgi:** Hoca, başlangıç değişkeni $\varepsilon$ üretiyorsa CNF'ye dönüştürülemeyeceğini düşündüğünü söyledi. Linz'deki CNF tanımında dilde $\varepsilon$ olmadığı varsayılır. Sipser'ın tanımında ise yalnız başlangıç değişkeni için $S \to \varepsilon$ kuralına izin verilir ve $S$ hiçbir kuralın sağ tarafında geçmez. Hangi tanımın kullanıldığına dikkat edin.
- **GNF'ye dönüştürme:** Derste gösterilen yöntem (terminaller için $T_a$, $T_b$ eklemek) yalnız her kural zaten bir terminalle başlıyorsa işe yarar. $S \to AB$ ya da sol özyinelemeli $A \to Aa$ gibi kurallarda önce sol özyineleme kaldırılmalı ve değişkenler sıralanarak yerine koyma yapılmalıdır. Kitaptaki "GNF'yi bulmak daha zordur" notu bu genel durum içindir.
- **Doğrulanan örnekler:** Yerine koyma, boşa giden değişken, birim üretim, yararsız üretim, CNF ve GNF örneklerinin hepsinde eski ve yeni gramerlerin aynı dili ürettiği Python ile denendi. Hepsi doğru çıktı.

**Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz. Ders İngilizce anlatılmıştır; altyazı da büyük ölçüde İngilizcedir.